Emotional-Trajectory Detection of Social Engineering

MVP + Evaluation Pipeline Checkpoint

Team 6: Sanchusri Kavitha Babu, Rahul Ajith Kumar, Sisvanthkumar Sathivadivel

Problem recap

We're detecting BEC and vishing social-engineering conversations as early as possible, before the harmful request (a wire transfer, a bank-detail change, a login code, a payroll redirect) is actually made, rather than only flagging a conversation after the fact. The idea is that a model tracking how emotional tone shifts turn by turn can flag a conversation earlier than one that only looks at the current message.

What's in scope for this checkpoint

Our full proposal has six scorers, three real external datasets, and several more shortcut checks and experiments. For this checkpoint we built and ran the subset below, and left the rest for later:

- **Kept now**: our four real escalation patterns (executive wire request, vendor bank-detail change, IT helpdesk vishing, payroll redirect), each paired with a matched legitimate “twin” conversation; real per-turn emotion features from the pretrained classifier named in our proposal; four of our six scorers (position-only, message-level emotion, running aggregate, and the trajectory GRU); the held-out pattern and held-out generator splits; and the opening-turn leakage shortcut check.
- **Built but not used in this run**: a real-data pipeline that downloads and parses the real Enron email corpus into genuine conversation threads, plus real ESConv and AnnoMI conversations, for the legitimate side of the data (`data/external_data.py`, `data/external/*.json`), along with a secondary FPR check against real Enron threads (`eval/evaluate.py`). We built and ran this end to end, then decided to keep this checkpoint synthetic-only for now; the code and downloaded data are in the repo and ready to switch back on.
- **Deferred**: real LLM-generated scam/twin conversations (still template-generated — no API key for an external model in this environment), the LLM-restyled authorship-check subset of Enron threads, the two text-based scorers and the DistilBERT reference ceiling, the rest of the shortcut-check suite, the slow-burn evaluation, and feature ablations.

1. Working core system / MVP

The pipeline has four stages, all in the attached repo (`scam_trajectory_mvp/`), and it runs end to end with zero errors:

- **Data generation** — builds matched scam/twin conversation pairs across our four patterns, varying length (8–16 turns) and ask position so turn index alone can't give it away. One pattern (payroll redirect) and one generator voice are held out entirely, for the generalization splits.
- **Emotion features** — every turn is scored by the pretrained classifier named in our proposal (`j-hartmann/emotion-english-distilroberta-base`, 7 emotion classes), giving a 15-dim feature per turn (the distribution, its change from the previous turn, and a speaker flag).

- **Four scorers** — position-only and message-level (emotion) as baselines, running aggregate (the TA-suggested baseline: running mean and running max of message-level scores so far, averaged together), and our main model: a 1-layer GRU with 64 hidden units over the per-turn emotion features, trained to output a running score at every turn.
- **Evaluation** — conversation-level accuracy/F1/AUC, plus the AT@FPR earliness metric from our proposal, on the in-distribution test split and all three held-out splits.

2. Data and evaluation pipeline

Data

Every conversation in this run — scam, matched twin, and legitimate — is still template-generated rather than pulled from a real LLM or real text, this environment has no API key for an external model. The patterns, roles, and matched-twin design all come directly from our proposal, and four simulated “generator voices” stand in for different LLMs until we wire in real calls (stubbed in `data/llm_backend.py`). We also built a real-data path for the legitimate side — actual Enron email threads parsed from the public corpus, plus real ESConv and AnnoMI conversations — and ran it end to end earlier (see `data/external_data.py`, `data/external/*.json`), but this checkpoint uses synthetic data throughout so the numbers below are a clean, consistent baseline; that real-data code is untouched and easy to switch back on.

Addressing the specific points from the proposal and TA feedback — **shortcut risk**: every scam scenario has a matched legitimate twin sharing the same roles, amount, and deadline, so the model can't just learn “LLM-written vs. normal” or “big dollar amount = scam.” **Generalization**: the payroll-redirect pattern and one generator voice are held out entirely from training, giving us three held-out test splits (unseen pattern, unseen generator, both). **Earliness metric**: AT@FPR exactly as scoped — calibrate a threshold on legitimate validation conversations at a fixed false-positive rate, then measure how many turns before the harmful request each scam conversation first crosses it.

Evaluation

For each of the four scorers, we report classification quality (accuracy/F1/AUC, using each conversation's max running score) and earliness (AT@FPR, calibrated separately at 5% and 10% FPR, both suggested in our proposal), on all four splits.

3. Baseline(s)

Three baselines, in increasing order of what they use: position-only (just the turn index, the strongest timing baseline from the PreScam paper our proposal builds on), message-level emotion (a single turn's emotion features, no memory of earlier turns), and running aggregate (the running mean and running max of message-level scores so far, averaged together — the TA's suggested baseline). The trajectory GRU is evaluated against all three.

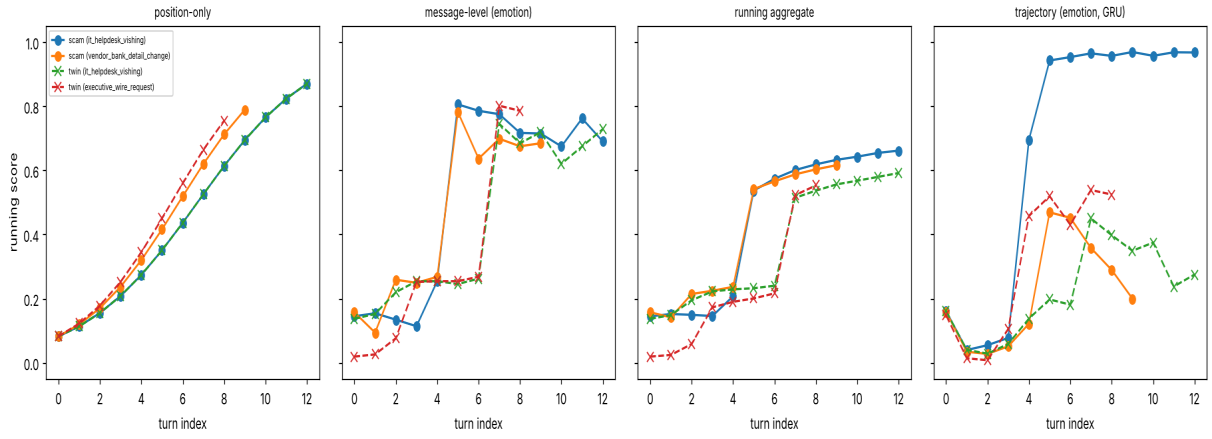
4. First experiment

We trained all four scorers on the synthetic training split, then evaluated on the in-distribution test split and all three held-out splits.

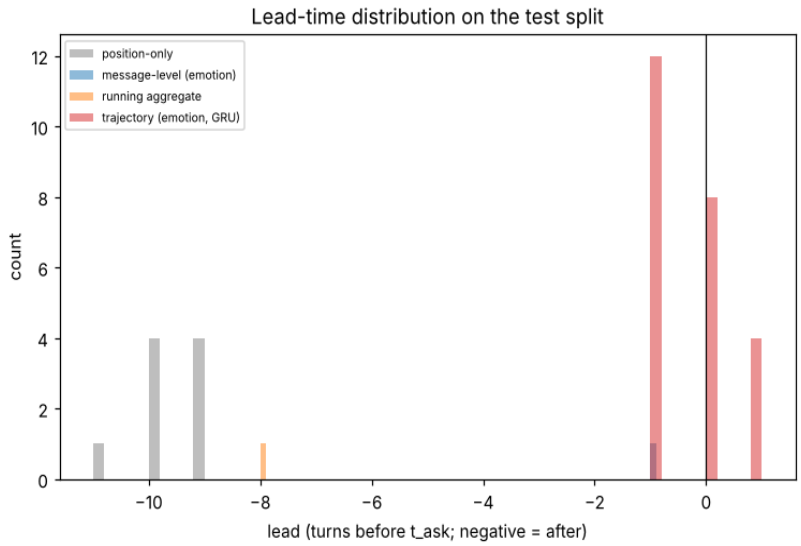
Scorer	Split	Accuracy	F1	AUC
Position-only	test	0.500	0.286	0.500
Position-only	heldout pattern	0.500	0.143	0.500
Position-only	heldout LLM	0.500	0.250	0.500
Position-only	heldout both	0.500	0.250	0.500
Message-level (emotion)	test	0.489	0.042	0.429
Message-level (emotion)	heldout pattern	0.500	0.000	0.646
Message-level (emotion)	heldout LLM	0.417	0.000	0.365
Message-level (emotion)	heldout both	0.500	0.000	0.649
Running aggregate	test	0.511	0.043	0.524
Running aggregate	heldout pattern	0.500	0.143	0.556
Running aggregate	heldout LLM	0.458	0.000	0.472
Running aggregate	heldout both	0.500	0.000	0.569
Trajectory (GRU)	test	0.733	0.667	0.828
Trajectory (GRU)	heldout pattern	0.500	0.000	0.750
Trajectory (GRU)	heldout LLM	0.708	0.588	0.910
Trajectory (GRU)	heldout both	0.500	0.000	0.757

Classification metrics use each conversation's max running score. The trajectory GRU clearly leads every baseline on every split (AUC 0.75–0.91); the three baselines sit at or near chance (AUC 0.36–0.65). See Section 5 for why the GRU's numbers moved up from an earlier synthetic-only run and what changed.

Opening-turn leakage check (can a simple classifier tell a scam conversation from its matched twin using only the first 2 turns?): **0.470** 5-fold CV accuracy, where 0.5 is chance and 1.0 would mean the generation process leaked the label into the rapport turns. This is close to chance, which is what we want — it means the matched-twin design is doing its job.



Running score per turn for two example scam conversations (solid, circle markers) vs. two matched-twin conversations (dashed, x markers), on the test split, for all four scorers.



Distribution of detection lead time (turns before the harmful request; negative means after it) on the test split, at the 5%-FPR threshold.

5. Preliminary results and what we're reading from them

Three things stand out from this run:

- **The three baselines sit at or near chance and never fire early.** Position-only, message-level, and running aggregate all land in the AUC 0.36–0.65 range (0.5 = coin flip) and show 0% early_recall on every split at both FPR targets we calibrated to (5%, 10%). None of them pick up a usable signal before the harmful ask.
- **The trajectory GRU now clearly separates scam from legitimate, and shows some real earliness in-distribution.** Its AUC is 0.75–0.91 across all four splits, beating every baseline everywhere. On the in-distribution test split specifically, early_recall is 9% at the 5%-FPR threshold and 27% at the 10%-FPR threshold (median lead of 1 turn when it fires early). That's new since we added class-balanced loss (a pos_weight term) to the GRU's training objective — before that change, every scorer including the GRU showed 0% early_recall everywhere.
- **But that earliness doesn't generalize yet.** On all three held-out splits (unseen pattern, unseen generator voice, both), the GRU's early_recall is still 0%, even though its classification AUC on those same splits is high (0.75–0.91). So it can tell a held-out scam conversation apart from a legitimate one by the end, but it hasn't learned to flag a *novel* escalation pattern or voice before the harmful ask — only the pattern/voices it was trained on. Most likely cause: our per-turn training label is 1 only at or after the ask turn, so the model never gets a training signal that earlier urgency/authority turns in an unfamiliar pattern belong to the same kind of escalation — worth trying next, by widening how we label the pre-ask turns during training.

The opening-turn leakage check (0.470, reported above) stays close to chance, which tells us the matched-twin design is doing its job: the generation process isn't leaking the scam/twin label into the early rapport turns.

Next steps

- Try widening the per-turn training label to include the authority/urgency turns right before the ask, and see whether that gives the GRU an earlier signal to learn from, especially on the held-out splits.

- Switch the real-data path back on (Enron/ESConv/AnnoMI legitimate conversations and the secondary FPR check against real Enron threads — built and validated earlier, see `data/external_data.py`) once we've decided this checkpoint's synthetic-only numbers are the baseline we want to compare against.
- Replace the template generator with real LLM-generated conversations, using our actual escalation patterns (stubbed and ready to wire in — `data/llm_backend.py`) — this needs an API key we don't have in this environment yet.
- Generate the LLM-restyled authorship-check subset: we already pulled and reserved 300 real Enron threads for this (`data/external/enron_authorship_source.json`), just haven't had an LLM rewrite them yet, same reason as above.
- Add the DistilBERT text-based scorers as the reference ceiling.
- Finish the rest of the shortcut-check suite (authorship probe, full position + correlation analysis, human realism audit) and add the slow-burn evaluation split.